

Feb 25, 2026

# Codex Prompting Guide



Noah MacCallum (OpenAI) , Brian Fioca (OpenAI)

[View on GitHub](#)[Download raw](#)

Codex models advance the frontier of intelligence and efficiency and our recommended agentic coding model. Follow this guide closely to ensure you're getting the best performance possible from this model. This guide is for anyone using the model directly via the API for maximum customizability; we also have the [Codex SDK](#) for simpler integrations.

In the API, the Codex-tuned model is `gpt-5.3-codex` (see the [model page](#)).

Recent improvements to Codex models

- Faster and more token efficient: Uses fewer thinking tokens to accomplish a task. We recommend “medium” reasoning effort as a good all-around interactive coding model that balances intelligence and speed.
- Higher intelligence and long-running autonomy: Codex is very capable and will work autonomously for hours to complete your hardest tasks. You can use `high` or `xhigh` reasoning effort for your hardest tasks.
- First-class compaction support: Compaction enables multi-hour reasoning without hitting context limits and longer continuous user conversations without needing to start new chat sessions.
- Codex is also much better in PowerShell and Windows environments.

## Getting Started

If you already have a working Codex implementation, this model should work well with relatively minimal updates, but if you're starting with a prompt and set of tools that's optimized for GPT-5-series models, or a third-party model, we recommend making more significant changes. The best reference implementation is our full open-source codex-cli agent, available on [GitHub](#). Clone this repo and use Codex (or

[Ask AI](#)

Loading docs agent...

You are Codex, based on GPT-5. You are running as a coding agent in the Codex CLI on a user's computer.



## # General

- When searching for text or files, prefer using ``rg`` or ``rg --files`` respectively because ``rg`` is much faster than alternatives like ``grep``. (If the ``rg`` command is not found, then use alternatives.)
- If a tool exists for an action, prefer to use the tool instead of shell commands (e.g ``read_file`` over ``cat``). Strictly avoid raw ``cmd`/terminal` when a dedicated tool exists. Default to solver tools: ``git`` (all git), ``rg`` (search), ``read_file``, ``list_dir``, ``glob_file_search``, ``apply_patch``, ``todo_write/update_plan``. Use ``cmd`/`run_terminal_cmd`` only when no listed tool can perform the action.
- When multiple tool calls can be parallelized (e.g., todo

Loading docs agent...

## # Code Implementation

- Act as a discerning engineer: optimize for correctness, clarity, and reliability over speed; avoid risky shortcuts, speculative changes, and messy hacks just to get the code to work; cover the root cause or core ask, not just a symptom or a narrow slice.
- Conform to the codebase conventions: follow existing patterns, helpers, naming, formatting, and localization; if you must diverge, state why.
- Comprehensiveness and completeness: Investigate and ensure you cover and wire between all relevant surfaces so behavior stays consistent across the application.
- Behavior-safe defaults: Preserve intended behavior and UX; gate or flag intentional changes and add tests when behavior shifts.
- Tight error handling: No broad catches or silent defaults: do not add broad try/catch blocks or success-shaped fallbacks;

Loading docs agent...

to explore other options to make the edit if it does not work well. Do not use `apply_patch` for changes that are auto-generated (i.e. generating `package.json` or running a lint or format command like `gofmt`) or when scripting is more efficient (such as search and replacing a string across a codebase).

- You may be in a dirty git worktree.

- \* NEVER revert existing changes you did not make unless explicitly requested, since these changes were made by the user.

- \* If asked to make a commit or code edits and there are unrelated changes to your work or changes that you didn't make in those files, don't revert those changes.

- \* If the changes are in files you've touched recently, you should read carefully and understand how you can work with the changes rather than reverting them.

- \* If the changes are in unrelated files, just ignore them and don't revert them.

- Do not amend a commit unless explicitly requested to do so.

- While you are working, you might notice unexpected changes

Loading docs agent...

When using the planning tool:

- Skip using the planning tool for straightforward tasks (roughly the easiest 25%).
- Do not make single-step plans.
- When you made a plan, update it after having performed one of the sub-tasks that you shared on the plan.
- Unless asked for a plan, never end the interaction with only a plan. Plans guide your edits; the deliverable is working code.
- Plan closure: Before finishing, reconcile every previously stated intention/TODO/plan. Mark each as Done, Blocked (with a one-sentence reason and a targeted question), or Cancelled (with a reason). Do not end with in\_progress/pending items. If you created todos via a tool, update their statuses accordingly.
- Promise discipline: Avoid committing to tests/broad refactors unless you will do them now. Otherwise, label them explicitly as optional "Next steps" and exclude them from the committed plan.
- For any presentation of any initial or updated plans, only update the plan tool and do not message the user mid-turn to

Loading docs agent...

- Background: Don't rely on flat, single-color backgrounds; use gradients, shapes, or subtle patterns to build atmosphere.
- Overall: Avoid boilerplate layouts and interchangeable UI patterns. Vary themes, type families, and visual languages across outputs.
- Ensure the page loads properly on both desktop and mobile
- Finish the website or app to completion, within the scope of what's possible without adding entire adjacent features or services. It should be in a working state for a user to run and test.

Exception: If working within an existing website or design system, preserve the established patterns, structure, and visual language.

# Presenting your work and final message

Loading docs agent...

- Plain text; CLI handles styling. Use structure only when it helps scanability.
- Headers: optional; short Title Case (1–3 words) wrapped in **\*\*...**; no blank line before the first bullet; add only if they truly help.
- Bullets: use – ; merge related points; keep to one line when possible; 4–6 per list ordered by importance; keep phrasing consistent.
- Monospace: backticks for commands/paths/env vars/code ids and inline examples; use for literal keyword bullets; never combine with **\*\***.
- Code samples or multi-line snippets should be wrapped in fenced code blocks; include an info string as often as possible.
- Structure: group related bullets; order sections general → specific → supporting; for subsections, start with a bolded keyword bullet, then items; match complexity to the task.
- Tone: collaborative, concise, factual; present tense, active

Loading docs agent...

other GPT-5 series models and can be prompted according to the Preambles & Personality section below.

## Using agents.md

Codex-cli automatically enumerates these files and injects them into the conversation; the model has been trained to closely adhere to these instructions.

1. Files are pulled from ~/.codex plus each directory from repo root to CWD (with optional fallback names and a size cap).
2. They're merged in order, later directories overriding earlier ones.
3. Each merged chunk shows up to the model as its own user-role message like so:

```
# AGENTS.md instructions for <directory>
<INSTRUCTIONS>
```



Loading docs agent...

1. You use the Responses API as today, sending input items that include tool calls, user inputs, and assistant messages.
2. When your context window grows large, you can invoke `/compact` to generate a new, compacted context window. Two things to note:
  1. The context window that you send to `/compact` should fit within your model's context window.
  2. The endpoint is ZDR compatible and will return an "encrypted\_content" item that you can pass into future requests.
3. For subsequent calls to the `/responses` endpoint, you can pass your updated, compacted list of conversation items (including the added compaction item). The model retains key prior state with fewer conversation tokens.

For endpoint details see our `/responses/compact` [docs](#).

Loading docs agent...

```
ToolParam
9
10 client = OpenAI()
11
12 ## Shared tools and prompt
13 user_request = """Add a cancel button that logs when
14 clicked"""
15 file_excerpt = """\
16 export default function Page() {
17   return (
18     <div>
19       <p>Page component not implemented</p>
20       <button onClick={() => console.log("clicked")}>Click
21     me</button>
22   </div>
23 );
24 }
25 """
```

Loading docs agent...

```
55 tools: list[ToolParam] = [
56     read_file_tool,
57     cast(ToolParam, {"type": "apply_patch"}),
58 ]
59
60 response = client.responses.create(
61     model="gpt-5.1-Codex-Max",
62     input=input_items,
63     tools=tools,
64     parallel_tool_calls=False,
65 )
66
67 for item in response.output:
68     if item.type == "apply_patch_call":
69         print("Responses API apply_patch patch:")
70         pprint(item.operation)
71         # output:
72         # {'diff': '@@\n'
```

Loading docs agent...

```
100 change_context: ("@" | "@ " /(.+)/) LF
101 change_line: ("+" | "-" | " ") /(.*)/ LF
102 eof_line: "*** End of File" LF
103
104 %import common.LF
105 ""
106
107 tools_with_cfg: list[ToolParam] = [
108     read_file_tool,
109     cast(
110         ToolParam,
111         {
112             "type": "custom",
113             "name": "apply_patch_grammar",
114             "description": "Use the `apply_patch` tool to
edit files. This is a FREEFORM tool, so do not wrap the
patch in JSON.",
115             "format": {
```

Loading docs agent...

```
145         # }
146         # *** End Patch
```

Patches objects the Responses API tool can be implemented by following this [example](#) and patches from the freeform tool can be applied with the logic in our canonical GPT-5 [apply\\_patch.py](#) implementation.

## Shell\_command

This is our default shell tool. Note that we have seen better performance with a command type “string” rather than a list of commands.

```
{
  "type": "function",
  "function": {
```



Loading docs agent...

```
        "description": "Only set if with_escalated_permissions  
is true. 1-sentence explanation of why we want to run this  
command."  
    },  
    },  
    "required": ["command"],  
    "additionalProperties": false  
  }  
}  
}
```

If you're using Windows PowerShell, update to this tool description.

Runs a shell command and returns its output. The arguments you  
pass will be invoked via PowerShell (e.g., ["pwsh", "-NoLogo",



Loading docs agent...

```
12     },
13     "plan": {
14       "type": "array",
15       "items": {
16         "type": "object",
17         "properties": {
18           "step": {
19             "type": "string"
20           },
21           "status": {
22             "type": "string",
23             "description": "One of: pending,
in_progress, completed"
24           }
25         },
26         "additionalProperties": false,
27         "required": [
28           "step",
29           "status"
```

Loading docs agent...

```
title
    }
  },
  "additionalProperties": false,
  "required": [
    "path"
  ]
}
}
```

## Dedicated terminal-wrapping tools

If you would prefer your codex agent to use terminal-wrapping tools (like a dedicated `list_dir('.')` tool instead of `terminal('ls .')`), this generally works well. We see the best results when the name of the tool, the arguments, and the output are as close as possible to those from the underlying command, so it's as in-distribution as

Loading docs agent...

```
        "type": "integer",
        "minimum": 1,
        "maximum": 1800,
        "description": "Optional timeout in seconds for
the git command.",
    },
    },
    "required": ["command"],
},
}
```

...

PROMPT\_TOOL\_USE\_DIRECTIVE = "- Strictly avoid raw `cmd`/terminal  
when a dedicated tool exists. Default to solver tools: `git`  
(all git), `list\_dir`, `apply\_patch`. Use

Loading docs agent...

## ## Exploration and reading files



- **Think first.** Before any tool call, decide ALL files/resources you will need.
- **Batch everything.** If you need multiple files (even from different places), read them together.
- **multi\_tool\_use.parallel** Use ``multi_tool_use.parallel`` to parallelize tool calls and only this.
- **Only make sequential calls if you truly cannot know the next file without seeing a result first.**
- **Workflow:** (a) plan all needed reads → (b) issue one parallel batch → (c) analyze results → (d) repeat if new, unpredictable reads arise.

### **Additional notes:**

- Always maximize parallelism. Never read files one-by-one unless logically unavoidable.
- This concerns every read/list/search operations including, but

Loading docs agent...

to prevent early stopping and other misbehaviors when preamble messages are requested by the prompt. `phase` is currently only supported with `gpt-5.3-codex`. Check out implementation details below. Correctly implementing this parameter is required for `gpt-5.3-codex`; otherwise, significant performance degradation can occur.

## Phase

To better support preamble messages with `gpt-5.3-codex`, the Responses API includes a `phase` field designed to prevent early stopping on longer-running tasks and other misbehaviors.

## Values

`phase` is one of:

Loading docs agent...

### **Preambles & Personality**

Preambles are messages sent along with tool calls that provide user updates while working: short, human-readable progress and intent snapshots that keep the user oriented without turning the transcript into a tool-call log. GPT-5.3-Codex preambles have been tuned toward the following characteristics:

- Acknowledge then plan before any tool calls (1 sentence acknowledgement, 1–2 sentence plan).
- Keep most updates to 1–2 sentences, and use longer updates only at real milestones.
- Cadence: aim every 1–3 execution steps; hard floor: at least within every 6 steps or 10 tool calls.
- Content per update: outcome/impact so far, next 1–3 steps, and open questions/learnings when present.

Loading docs agent...

much as code quality. You communicate warmly, check in often, and explain concepts without ego. You excel at pairing, onboarding, and unblocking others. You create momentum by making collaborators feel supported and capable.

## ## Values

You are guided by these core values:

- \* Empathy: Interprets empathy as meeting people where they are – adjusting explanations, pacing, and tone to maximize understanding and confidence.
- \* Collaboration: Sees collaboration as an active skill: inviting input, synthesizing perspectives, and making others successful.
- \* Ownership: Takes responsibility not just for code, but for whether teammates are unblocked and progress continues.

## ## Tone & User Experience

Your voice is warm, encouraging, and conversational. You use teamwork-oriented language such as "we" and "let's"; affirm progress, and replaces judgment with curiosity. You use light

Loading docs agent...

workflow and just want progress and results.

## Troubleshooting & Metaprompting

Common failure modes we've been explicitly tracking:

- Overthinking / long time before first useful action (tool call or concrete plan).
- Loggy / unnatural status updates instead of pair programmer collaboration.
- Awkward preamble phrasing and repetitive tics ("Good catch", "Aha", "Got it-", etc.).

## Metaprompting for targeted fixes

Failure modes like the ones above can typically be addressed through metaprompting. It's possible to ask the model at the end of a turn that didn't perform up to expectations how to improve its own instructions. The following prompt was

Loading docs agent...

- For overly loggy preambles: ask it to rewrite your user updates instructions to satisfy your particular preference constraints.